CS-112
Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Today we will talk about!

- Constructors
- Types of Constructors
- Constructor Overloading
- Copy Constructor
- Destructors
- Finalize () Method

Constructors In Java

Constructors

- A constructor is a special block of code that runs automatically when you create a new object. Its main purpose is to initialize the object—that is, to set up initial values for the object's variables.
- It has the same name as the class.
- It doesn't have a return type, not even void.
- It runs automatically when you create an object using new.
- You can create multiple constructors in the same class (constructor overloading).
- Constructors are typically public.

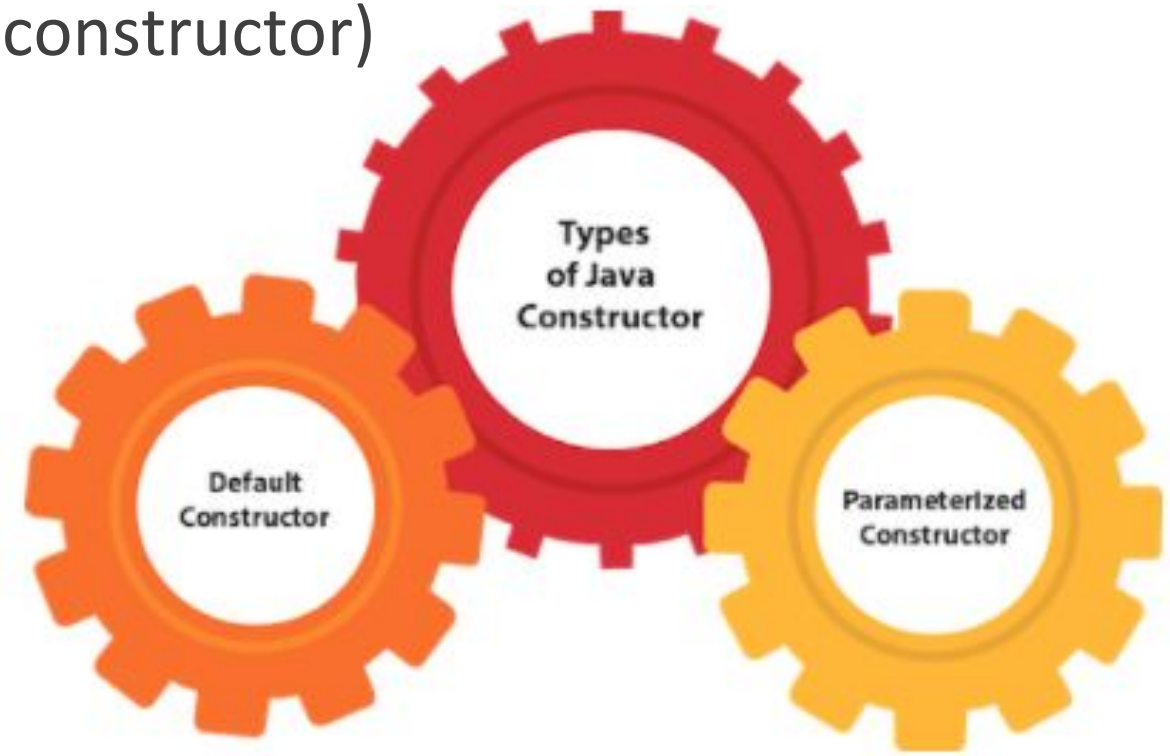
Example

```
class Car {
    String color;
    // Constructor
    Car(String c) {
        color = c;
    }
}
// Creating an object
Car myCar = new Car("Red"); // Constructor runs here
```

Types of Java Constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor



The Default Constructor

A default constructor is a constructor that:

- Takes no arguments (nothing inside the parentheses).
- Is used to create an object with default values.

Types- Default Constructor

Automatically provided by Java

- If **you don't write any constructor**, Java adds one for you.
- This is called the **default constructor**.
- It does **nothing special**, just lets you create the object.

```
class Car {
    string color;
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Java automatically provides a default constructor
        System.out.println(myCar.color); // This will print: null (because no value was set)
    }
}
```

```
//Let us see another example of default constructor
//which displays the default values
class Student3{
int id;
String name;
//method to display the value of id and name
void display(){System.out.println(id+" "+name);}

public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the object
s1.display();
s2.display();
}
}
```

Output

0 null

0 null

Types-
Default Constructor

Custom default constructor

- You can also **write your own** default constructor if you want to set some starting values.

```
class Car {
    String color;

    // Default constructor
    Car() {
        color = "Red"; // Set a car color
    }
}
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Your
constructor runs
        System.out.println(myCar.color); // This
will print: Red
    }
}
```

Question

- **What is the purpose of a default constructor?**

  The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also

```java
//Java Program to demonstrate the use of the parameterized constructor.
class Student4{
    int id;
    String name;
    //creating a parameterized constructor
    Student4(int i,String n){
    id = i;
    name = n;
    }
    //method to display the values
    void display(){System.out.println(id+" "+name);}

    public static void main(String args[]){
    //creating objects and passing values
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    //calling method to display the values of object
    s1.display();
    s2.display();
   }
}
```

Task

A company needs to store and manage employee information, such as ID and name. When an employee joins, an Employee object is created with a constructor that sets these values. The employee's details are private and cannot be accessed directly. The company uses getter methods to view the details and setter methods to update them if needed. This ensures that employee information is encapsulated and can be safely modified through methods.

```java
// Main class
public class FirstJavaCode {
    public static void main(String[] args) {
        // Creating an Employee object using
constructor
        Employee emp = new Employee(45,
"Smith");

        // Displaying employee details
        System.out.println("Id of Employee
is: " + emp.getId());
        System.out.println("Name of Employee
is: " + emp.getName());

        // Changing the id and name using
setter methods
        emp.setId(50);
        emp.setName("John");

        // Displaying updated employee
details
        System.out.println("\nUpdated
Employee Details:");
        System.out.println("Id of Employee
is: " + emp.getId());
        System.out.println("Name of Employee
is: " + emp.getName());
    }
}
```

```java
class Employee {
    // Private fields
    private int id;
    private String name;

    // Constructor to set ID and name when
creating the object
    Employee(int i, String myName) {
        id = i;
        name = myName;
    }

    // Getter method for id
    public int getId() {
        return id;
    }

    // Getter method for name
    public String getName() {
        return name;
    }

    // Setter method for id
    public void setId(int id) {
        this.id = id;
    }

    // Setter method for name
    public void setName(String name) {
        this.name = name;
    }
```

Constructor Overloading in Java

1. Constructor overloading in Java is a technique of having more than one constructor with different parameter lists.
2. They are arranged in a way that each constructor performs a different task.
3. They are differentiated by the compiler by the number of parameters in the list and their types.

```
/Java program to overload constructors
class Student5{
    int id;
    String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n){
    id = i;
    name = n;
    }
    //creating three arg constructor
    Student5(int i,String n,int a){
    id = i;
    name = n;
    age=a;
    }
    void display(){System.out.println(id+" "+name+" "+age);}

    public static void main(String args[]){
        Student5 s1 = new Student5(111,"John");
        Student5 s2 = new Student5(222,"Smith",25);
        s1.display();
        s2.display();
       }
}
```

Output:

111 John 0

222 Smith 25

Java Copy Constructor

A copy constructor in Java is a special type of constructor used to create a new object by copying the values of an existing object of the same class.

It allows you to create a new object that is an exact copy of an already existing object.

A copy constructor takes an object of the same class as a parameter.

It creates a new object with the same values as the passed object.

```
ClassName(ClassName obj) {
   // Copy the values of obj to the new object
}
```

```
//Java program to initialize the values from one object
 to another object.
class Student6{
    int id;
    String name;
    //constructor to initialize integer and string
    Student6(int i, String n){
    id = i;
    name = n;
    }
    //constructor to initialize another object
    Student6(Student6 s){
    id = s.id;
    name =s.name;
    }
    void display(){System.out.println(id+" "+
name);}
    public static void main(String args[]){

// Creating the first Student6 object
    Student6 s1 = new Student6(111,“Ali")
;
// Creating the second Student6 object
using the copy constructor
    Student6 s2 = new Student6(s1);
    s1.display();
    s2.display();
   }
}
```

Difference Between Constructor And Method In Java

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as the class name.

Destructors In Java

Destructors

- In Java, when we create an object of the class it occupies some space in the memory (heap).
- If we do not delete these objects, it remains in the memory and occupies unnecessary space that is not upright from the aspect of programming.
- To resolve this problem, we use the **destructor.**
- The **destructor** is the opposite of the constructor.
- The constructor is used to initialize objects while the destructor is used to delete or destroy the object that releases the resource occupied by the object.

Destructors (Cont...)

Remember that **there is no concept of destructor in Java**.

In place of the destructor, Java provides the **garbage collector** that works the same as the destructor.

The garbage collector is a program (thread) that runs on the JVM.

It automatically deletes the unused objects (objects that are no longer used) and free-up the memory.

Destructors cannot be overloaded

How does destructor work?

- When the object is created it occupies the space in the heap. These objects are used by the threads.
- If the objects are no longer is used by the thread it becomes eligible for the garbage collection. The memory occupied by that object is now available for new objects that are being created.
- It is noted that when the garbage collector destroys the object, the JRE calls the finalize() method to close the connections such as database and network connection.

Java finalize() Method

It is difficult for the programmer to forcefully execute the garbage collector to destroy the object. But Java provides an alternative way to do the same. The Java Object class provides the **finalize()** method that works the same as the destructor. The syntax of the finalize() method is as follows:

Syntax:

```
protected void finalize throws Throwable()
{
//resources to be close
}
```

Cont..

It is not a destructor but it provides extra security. It ensures the use of external resources like closing the file, etc. before shutting down the program. We can call it by using the method itself or invoking the method **System.runFinalizersOnExit(true)**.

It is a protected method of the Object class that is defined in the java.lang package.

It can be called only once.

We need to call the finalize() method explicitly if we want to override the method.

The gc() is a method of JVM executed by the Garbage Collector. It invokes when the heap memory is full and requires more memory for new arriving objects.

Except for the unchecked exceptions, the JVM ignores all the exceptions that occur by the finalize() method.

Cont…

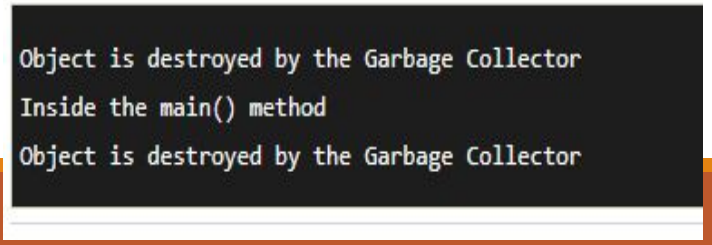
finalize() is considered deprecated in Java 9 and later because it is not reliably called and may lead to memory leaks.

Use try-with-resources or explicit cleanup methods like close() when managing resources (e.g., files, database connections).

Example GC

```
public class DestructorExample {
    public static void main(String[] args) {
        DestructorExample de = new DestructorExample(); // create object
        de.finalize(); // manually call finalize (not recommended)
        de = null; // make object eligible for garbage collection
        System.gc(); // request Java garbage collector to run
        System.out.println("Inside the main() method");
    }
    protected void finalize() {
        System.out.println("Object is destroyed by the Garbage Collector");
    }
}
```

Output



(Example GC)

```
public class DestructorExample {
    public static void main(String[] args) {
        DestructorExample de = new DestructorExample(); // create object
        de = null; // object is now eligible for garbage collection
        System.gc(); // request garbage collection
        System.out.println("Inside the main() method");
    }

    protected void finalize() {
        System.out.println("Object is destroyed by the Garbage Collector");
    }
}
```

Possible GC Outputs

Possible Output (if GC runs and calls finalize())

Object is destroyed by the Garbage Collector

Inside the main() method

The order is not guaranteed — you might also see:

Inside the main() method

Object is destroyed by the Garbage Collector

If the garbage collector doesn't run, then finalize() won’t be called at all.

Inside the main() method

When is destructor called?

A destructor function is called automatically when the object goes out of scope:

(1) the function ends
(2) the program ends
(3) a block containing local variables ends
(4) a delete operator is called

Advantages of Destructor

- It releases the resources occupied by the object.
- No explicit call is required, it is automatically invoked at the end of the program execution.
- It does not accept any parameter and cannot be overloaded.

How destructors are different from a normal member function?

- Destructors have same name as the class preceded by a tilde (~)
- Destructors don’t take any argument and don’t return anything

Sequence of Calls

- Constructors and destructors are called automatically
- Constructors are called in the sequence in which object are declared
- Destructors are called in reverse order

Class Task

Write a program to demonstrate the use of copy constructor.

Write a program to demonstrate the use of “this” pointer using destructors.